

```

LIST OF
NULL
OCTET STRING
SIGNED INTEGER
STRING
STRUCTURE
UNSIGNED INTEGER

```

```

type-ref:
    type-name
    scoped-type-name

```

```

qualifier:
    tag

```

A type definition associates a name (**type-name**) with a schema construct representing a TLV type or **pseudo-type**. The given name serves as a descriptive title for the type, as well as a means to refer to the type from elsewhere in the schema.

Type definitions (**type-def**) are often used to describe TLV types that appear directly in some form of communication. For example, a type definition MAY define the structure of data carried within the payload of a message. Some type definitions may be used to define general purpose TLV constructs which are then employed in the definitions of other types.

The type (**type-name**) associated with a type definition MAY be any one of the available TLV types or **pseudo-type**. Alternatively, a type definition MAY contain a scoped type (**scoped-type-name**) referring to another type definition appearing elsewhere in the schema. This form is referred to as a type reference (**type-ref**). The ordering of type definitions and type references within a schema is unimportant, implying that a type reference MAY refer to a type that is defined later in the schema.

A **tag** qualifier MAY be applied to the name within a type definition to associate a default tag with that name. The default tag will be used in an encoding of the type whenever an explicit tag has not been given.

B.2.2. FIELD GROUP Definition (field-group-def)

FIELD GROUP Definition Syntax

```

field-group-name => FIELD GROUP { field-group-members }

field-group-members:
    field-group-member
    field-group-members, field-group-member

field-group-member:
    identifier [ id-qualifier ] : type-or-ref    // field definition

```